

IG.com Homepage Migration - Developer Guide

Date: September 15, 2026

Adobe

Table of Contents

IG.com Homepage Migration - Developer Guide	3
Quick Reference	3
Architecture Overview	3
Local Development Setup	6
Design System	7
Blocks Reference	8
Universal Editor Models	9
Content Import Pipeline	10
JCR Generation Pipeline	10
Scripts & Integrations	11
Common Development Tasks	11
Environments	12
Git Workflow	12
Troubleshooting	12
Resources	13
Support Contacts	13

IG.com Homepage Migration - Developer Guide

This guide documents the technical implementation of the IG.com homepage migration to AEM Edge Delivery Services. It is intended for developers taking over maintenance and extension of the project. The project is an **xwalk / Universal Editor** implementation based on the AEM boilerplate, with a custom brand design system, eight custom homepage blocks, a content import pipeline, and a JCR generation pipeline for ingesting the migrated page into AEM author.

Quick Reference

Resource	URL
Code Repository	agnesl-adobe/iggroup (GitHub)
Preview	https://main-iggroup-agnesl-adobe.aem.page/
Live	https://main-iggroup-agnesl-adobe.aem.live/
Local Dev	http://localhost:3000
Config Service Org	agnesl-adobe (repoless — 9 sites share this org)
Content Source	AEM author (author-p179457-e1900808.adobe.aemcloud.com) via franklin.delivery (markup mount-point)

This is an **xwalk (Universal Editor / Crosswalk)** project: content is authored in AEM author against the Universal Editor and delivered through the `franklin.delivery` pipeline, rather than from Document Authoring or SharePoint. The Config Service org (`agnesl-adobe`) hosts multiple sites (repoless setup).

Architecture Overview

Tech Stack

- Vanilla JavaScript (ES6+), no framework
- CSS3 with Custom Properties (design tokens)
- No build step for page rendering — files are served directly
- Universal Editor authoring; content delivered via the AEM author markup pipeline
- A small JSON aggregation build (`npm run build:json`) for Universal Editor component definitions

Project Structure

```
├─ blocks/                # Block library (8 custom homepage blocks + boilerplate/WKND blocks)
├─ models/                # Universal Editor model sources (_component-*.json)
├─ component-models.json  # Aggregated UE models (generated by build:json)
├─ component-definition.json # Aggregated UE definitions (generated)
├─ component-filters.json # Aggregated UE filters (generated)
├─ scripts/
├─ aem.js                 # Core library (DO NOT MODIFY)
```

— scripts.js	# Custom page decoration + E-L-D orchestration
— delayed.js	# Adobe Target + custom library / tweetable loading
— utils.js	# Shared utilities (language, path, dynamic media)
— dom-helpers.js	# DOM builder helpers (a, span, i, picture, ...)
— editor-support*.js	# Universal Editor in-context editing support
— styles/	
— styles.css	# Critical styles, IG design tokens, heading scale, pill CTAs,
section variants	
— brand.css	# IG.com brand tokens + Matter SQ @font-face (IG CDN)
— fonts.css	# Boilerplate font definitions
— lazy-styles.css	# Non-critical global styles (empty placeholder)
— tools/importer/	# Content import pipeline (parsers, transformers, import script)
— jcr/	# JCR XML generation + installable FileVault package
— content/	# Migrated page source (en.plain.html) + this guide
— icons/	# SVG icons

Three-Phase Loading (E-L-D)

Edge Delivery Services uses a strict Eager-Lazy-Delayed loading strategy to achieve a Lighthouse score of 100. Every file and script must be placed in the correct phase. The orchestration lives in `scripts/scripts.js` (`loadEager` → `loadLazy` → `loadDelayed`).

Phase	Purpose	What Loads	Performance Impact
Eager	Render above-the-fold content as fast as possible (LCP)	<code>styles/styles.css</code> , first section's blocks, <code>scripts/scripts.js</code> , <code>decorateMain</code>	Blocks LCP — keep minimal
Lazy	Load remaining page content after first paint	Remaining blocks, header, footer, <code>lazy-styles.css</code> , fonts	Runs after Eager completes — safe for non-critical UI
Delayed	Load non-essential third-party scripts	<code>scripts/delayed.js</code> (Adobe Target, custom js-files, tweetable links)	Runs ~3s after page load — never blocks rendering

Rules for developers:

- **Never add third-party scripts to `scripts.js`** — they block LCP. Always use `delayed.js`.
- **Never load fonts eagerly** — fonts load lazily to avoid render-blocking. The Matter SQ web font is declared in `brand.css` with `font-display: swap`.
- Blocks in the first section load eagerly; all others load lazily. This is automatic based on DOM position.
- The header and footer are loaded in the lazy phase via `loadHeader()` / `loadFooter()` from `aem.js`.

Key Files & How They Connect

File	Role	Connects To
<code>scripts/aem.js</code>	Core library — <code>loadHeader</code> / <code>loadFooter</code> , <code>decorateSections</code> , <code>decorateBlocks</code> , <code>loadSection(s)</code> , <code>loadCSS</code> , <code>createOptimizedPicture</code> , <code>sampleRUM</code> , <code>toClassName</code> , <code>toCamelCase</code> . DO NOT MODIFY.	Imported by <code>scripts.js</code> and blocks

File	Role	Connects To
scripts/scripts.js	Entry point — orchestrates E-L-D (LoadEager → loadLazy → loadDelayed), decorateMain, buildAutoBlocks, moveInstrumentation (used by blocks for UE instrumentation), language detection via getLanguage.	Imports from aem.js, utils.js, dom-helpers.js, experiment-load.js
scripts/delayed.js	Adobe Target loader (at-lsig.js, property 549d426b-...), embedCustomLibraries (loads URLs from js-files metadata), tweetable Twitter links. Skipped on localhost and inside UE canvas.	Called by scripts.js in delayed phase
scripts/utils.js	Shared helpers: getLanguage, setPageLanguage, getPathDetails, computeLocalizedUrl, createSource, dynamicMediaAssetProcess, isInternalPage, getHostname. Carries WKND-derived multi-language scaffolding (PATH_PREFIX = / language-masters).	Imported by scripts.js, delayed.js, blocks
styles/brand.css	IG.com brand tokens (--brand-*) + Matter SQ @font-face from IG CDN (a.c-dn.net). Overrides the var(--brand-*, fallback) defaults in styles.css.	Referenced by styles.css token layer
styles/styles.css	Critical CSS: IG design tokens, IG heading scale, global pill-button (red CTA) overrides, .section.accent/.section.grey variants.	Consumed by all blocks and pages
blocks/{name}/{name}.js	Block logic — exports default function decorate(block). Auto-loaded when the block class appears in DOM.	May import createOptimizedPicture from aem.js, moveInstrumentation from scripts.js
models/_component-*.json	Universal Editor model/filter/definition sources. Aggregated into root component-*.json via npm run build:json.	Drives UE authoring dialogs and allowed-component nesting

Execution flow: page load → scripts.js → loadEager() (first section + eager blocks, decorateMain) → loadLazy() (remaining blocks, header, footer, fonts) → loadDelayed() (loads delayed.js → Adobe Target + custom libs)

Local Development Setup

Prerequisites

- Node.js (no `.nvmrc` or engines pinned — use current LTS)
- AEM CLI: `npm install -g @adobe/aem-cli`

Setup Steps

```
# Clone repository
git clone https://github.com/agnesl-adobe/iggroup.git
cd iggroup

# Install dependencies
npm install

# Start local server
aem up
```

Local Server — Repoless / Multi-Site

This org (`agnesl-adobe`) is a **repoless / multi-site** Config Service setup — nine sites share the org. When starting the local server you must specify the site's preview URL so the AEM CLI resolves content for the correct site:

```
aem up --pages-url=https://main--iggroup--agnesl-adobe.aem.page
```

Without `--pages-url`, local preview may fail or show content from the wrong site.

- URL: `http://localhost:3000`
- Auto-reload on file changes
- Uses preview content by default

Universal Editor JSON Build

After editing any model source under `models/`, regenerate the aggregated component JSON that the Universal Editor consumes:

```
npm run build:json
```

This runs three merge steps and rewrites root `component-models.json`, `component-definition.json`, and `component-filters.json`. **These aggregated files are generated artifacts — edit the `models/_component-*.json` sources, never the root files directly.**

Linting

```
npm run lint      # runs lint:js (eslint) + lint:css (stylelint)
npm run lint:js
npm run lint:css
```

Design System

The design system is split across two files: `styles/brand.css` (IG brand tokens + web font) and `styles/styles.css` (the token layer that references brand tokens with fallbacks, plus the IG heading scale, pill CTAs, and section variants). The `var(--brand-*, fallback)` pattern in `styles.css` means `brand.css` can override the whole palette without touching the consuming rules.

Fonts

The IG **Matter SQ** web font is loaded via `@font-face` in `brand.css` directly from IG's CDN (https://a.cdn.net/z/*.woff), across five weights (300, 400, 500, 600, 700), all with `font-display: swap`. Every font-family token maps to Matter SQ.

Family	Weights	Usage
Matter SQ (IG CDN)	300, 400, 500, 600, 700	Body and headings (all <code>--*font-family</code> tokens)
arial, sans-serif	—	Fallback while web font loads / on failure

CSS Custom Properties

Brand Colors (`brand.css`)

```
--brand-theme-color: #e61e1e; /* red — pill CTA fill */
--brand-link-color: #065bdb; /* IG blue — inline links */
--brand-link-hover-color: #0446aa;
--brand-accent-panel-color: #c7d9fd; /* accent panel (trust stats) */
--brand-background-color: #fff;
--brand-dark-color: #000;
--brand-light-color: #f5f5f5; /* grey section background */
--brand-footer-background-color: #000;
```

Typography Tokens (`styles.css`)

```
--body-font-family: "Matter SQ", arial, sans-serif;
--heading-font-family: "Matter SQ", arial, sans-serif;
```

IG Heading Scale (responsive — desktop values at ≥900px)

```
--heading-font-size-xxl: 32px; /* mobile */ → 42px (desktop)
--heading-font-size-xl: 28px → 36px
--heading-font-size-l: 24px → 30px
--heading-font-size-m: 20px → 22px
--body-font-size-m: 22px (mobile) → 18px (desktop)
```

Spacing & Layout

```
--global-gutter: 1.5rem;
--global-section-padding: 5rem;
--spacing-small: 1rem; --spacing-medium: 2rem; --spacing-large: 2.5rem;
--nav-height: 64px;
--border-radius-large: 1rem;
```

Breakpoints

This project uses **range-syntax media queries** (`width <= / >=`), not the standard boilerplate min-width set. Note the primary desktop breakpoint is 900px, with 768px and 1024px used for intermediate layout adjustments.

Query	Usage
<code>(width <= 768px)</code>	Mobile / small-tablet adjustments
<code>(width >= 768px)</code>	Tablet and up
<code>(width >= 900px)</code>	Primary desktop token switch (heading scale, body size)
<code>(width >= 1024px) / (width <= 1024px)</code>	Large-desktop / below-large adjustments
<code>print</code>	Print styles

Global CTA Style (Pill Buttons)

The boilerplate's rounded-rect buttons with Font Awesome chevrons are overridden globally in `styles.css`:

- **Primary CTA**: solid red (`--brand-theme-color`) pill, `border-radius: 72px`, white text, no `::after` chevron.
 - **Secondary CTA** (`.cta-button-secondary / .button-container.secondary`): transparent with dark outline, inverts on hover.
 - **Inline links** (`main a`): IG blue (`--link-color`), underline on hover, no arrow.

These overrides use `!important` to win over boilerplate specificity — be aware when adding new button styles.

Section Style Variants

Two section styles are wired both in CSS and in the Universal Editor section model (see Models below):

Variant	Applied via	Effect
<code>.section.accent</code>	Section style option accent	Background <code>--brand-accent-panel-color</code> (<code>#c7d9fd</code>), 56px vertical padding, rounded inner panel. Used on the trust-stats section.
<code>.section.grey</code>	Section style option grey	Background <code>--brand-light-color</code> , 56px vertical padding, rounded inner panel. Used on the "Join IG" section.
<code>.section.highlight</code>	Boilerplate default	Retained from boilerplate

Blocks Reference

Eight custom blocks were built for the homepage. Each has `.js`, `.css`, `metadata.json`, `README.md`, and a Universal Editor model (`_<name>.json`). They are forked from boilerplate base blocks (hero, carousel, cards, columns) rather than being option-classes on those bases — this matters for the columns family (see JCR note below). Each block imports `createOptimizedPicture` from `aem.js`; the cards blocks also import `moveInstrumentation` from `scripts.js` to preserve Universal Editor instrumentation on rebuilt DOM.

Block	Base	Purpose	Key Behaviour / Gotchas
hero-product	hero	Product-intro hero: centered heading, copy, CTA row, chat link, large product image below	Normalises authored order so text always renders before media regardless of how cells were authored

Block	Base	Purpose	Key Behaviour / Gotchas
carousel-prices	carousel	Live-prices ticker — horizontally scrolling market tiles	Each authored row → one tile; inspects the change cell's sign to colour gains blue / losses red
cards-markets	cards	Market-category grid: icon + title + description per card	First cell → cards-markets-icon; responsive multi-column grid; uses moveInstrumentation
cards-stats	cards	Statistics row: large figure + supporting label	First cell → cards-stats-figure, second → cards-stats-label; wraps on small screens; uses moveInstrumentation
columns-promo	columns	Side-by-side promotional panels (imagery, heading, copy, "Learn more")	Adds columns-promo-{n}-cols; picture-only columns flagged as media panels
columns-feature	columns	Image + feature content side by side (screenshot vs. benefit checklist + CTA)	Adds columns-feature-{n}-cols; single-picture columns flagged columns-feature-media
columns-links	columns	Two audience columns, each image + sub-heading + arrow link list	Adds columns-links-{n}-cols; picture columns flagged has-media
columns-steps	columns	Numbered steps beside a lifestyle image + CTA	Adds columns-steps-{n}-cols; single-picture columns flagged columns-steps-media

Important — columns family in JCR: The four `columns-*` blocks were generated as **forked blocks**, not as option-classes on the base `columns` block. They render correctly in Edge Delivery Services, but when serialized to JCR they collapse to AEM's native `columns` component **without a per-variant class**. Structure and content survive; the variant-specific brand CSS does not hook. To fix in AEM, model them as a `classes` option on the base `columns` block. This is a known follow-up (see JCR section).

Universal Editor Models

Models are authored under `models/_component-*.json` and aggregated into the root `component-*.json` files via `npm run build:json`. Customizations made during migration:

- **_component-filters.json** — Container→child filters added so the custom blocks are allowed inside sections and their children inside the blocks. The four `columns-*` variants each allow a `column` child; the three `cards-*` and `carousel-prices` allow a `card` child; all eight blocks were added to the section filter's allowed-component list.
- **_component-models.json** — `accent` and `grey` options added to the section style model (these drive the `.section.accent` / `.section.grey` CSS variants), plus models for the eight custom blocks.
- After any model edit, run `npm run build:json` to regenerate the root aggregated files. Do not hand-edit the root `component-*.json`.

Standard boilerplate models (`_page.json`, `_section.json`, `_button.json`, `_image.json`, `_text.json`, `_title.json`) are retained largely as-is (the section model was extended with the `accent/grey` options).

Content Import Pipeline

Located under `tools/importer/`, this pipeline scrapes/transforms the source IG.com homepage into Edge Delivery Services block tables. It is a development-time tool, not part of page rendering.

Component	Files	Purpose
Parsers	<code>parsers/{hero-product, carousel-prices, cards-markets, cards-stats, columns-promo, columns-feature, columns-links, columns-steps}.js</code>	One parser per custom block variant — maps source DOM to the block's table structure
Transformers	<code>transformers/ig-cleanup.js</code> , <code>transformers/ig-sections.js</code>	<code>ig-cleanup</code> strips source-site cruft; <code>ig-sections</code> splits content into Edge Delivery Services sections and applies section styles
Import script	<code>import-home.js</code> (+ bundled <code>import-home.bundle.js</code>)	Orchestrates the import for the homepage
Config	<code>page-templates.json</code> , <code>urls-home.txt</code>	Template/block mapping and target URL list
Reports	<code>reports/en.report.json</code> , <code>reports/import-home.report.xlsx</code>	Import run output/QA

JCR Generation Pipeline

Because this is an xwalk project, the migrated page must exist in AEM author as JCR content (it is not delivered from git). The pipeline (`jcr/`) converts the migrated markup into installable JCR XML.

Flow: `content/en.plain.html` → `markdown` → JCR XML via `@adobe/helix-md2jcr` (using this project's `component-models.json`, `component-definition.json`, `component-filters.json`).

- **Output package:** `jcr/iggroup-en-homepage-1.0.0.zip` — an installable FileVault content package.
- **Target path:** `/content/iggroup/language-masters/en`
- **Contents:** 11 sections, all 8 homepage block variants, section styles (accent on trust-stats, grey on Join IG), and page metadata. The package carries the full page chain (`iggroup` → `language-masters` → `en`, all `cq:Page`) so it builds the hierarchy from scratch.
- **Install:** AEM **Package Manager** (`/crx/packmgr` → Upload Package → Install), or `vlt/oakpal` CLI. Package sources live under `jcr/package-build/` (`META-INF/vault/` + `jcr_root/`). Re-zip `package-build` contents (with `META-INF/` and `jcr_root/` at archive root) after editing `.content.xml`.
- After install, `admin.hlx.page` preview + publish surfaces the page to `*.aem.page` / `*.aem.live`.

Known follow-up: The four `columns-*` variants serialize to AEM's native `columns` component without a per-variant class (they are forked blocks, not option-classes). To hook the variant brand CSS in AEM, re-model them as a `classes` option on the base `columns` block. See `jcr/README.md`.

Scripts & Integrations

delayed.js (Delayed Phase)

- **Adobe Target** — loads `scripts/at-lsig.js` with property `549d426b-0bcc-be60-ce27-b9923bfcad4f`. Skipped on localhost and inside the Universal Editor canvas (`/canvas/`).
- **embedCustomLibraries** — reads a comma-separated `js-files` page-metadata value and loads each URL via `loadScript`. Lets authors attach page-specific external JS.
- **buildTwitterLinks** — decorates `<tweetable>` markup into Twitter share links (WKND-derived feature).
- No render-critical code runs here — everything is post-LCP by design.

utils.js (Multi-language Scaffolding)

Carries WKND-derived multi-language/path utilities (`PATH_PREFIX = /language-masters`, `getLanguage`, `computeLocalizedUrl`, `fetchLanguageNavigation`) and dynamic-media helpers. The homepage migration targets `/en`; these utilities are available if the site expands to more languages.

Common Development Tasks

Add a New Block

1. Create `blocks/{name}/{name}.js` and `.css`, plus `metadata.json`, `README.md`, and UE model `models/_{name}.json`.
2. Implement export default function `decorate(block)`.
3. Add the block to `models/_component-filters.json` (section allowed-components, plus any child filter) and add its model to `models/_component-models.json`.
4. Run `npm run build:json` to regenerate the root aggregated JSON.
5. Test locally: `aem up --pages-url=https://main--iggroup--agnesl-adobe.aem.page`.

Modify Brand Tokens

1. Edit `styles/brand.css` (`--brand-*` variables and Matter SQ @font-face).
2. `styles.css` consumes these via `var(--brand-*, fallback)` — no change needed there for a pure token swap.
3. Test across sections (accent/grey) and CTA states.

Add a Section Style Variant

1. Add the CSS rule (`main .section.{name}`) in `styles.css`.
2. Add the option to the section style model in `models/_component-models.json`.
3. Run `npm run build:json`.

Add Analytics / Marketing Tool

1. Add to `scripts/delayed.js` (never `scripts.js`).
2. Verify delayed loading in the Network tab (~3s after load).

Re-generate / Re-ingest the Migrated Page

1. Update `content/en.plain.html` (or re-run the importer under `tools/importer/`).
2. Regenerate JCR XML via `@adobe/helix-md2jcr`, rebuild `jcr/package-build` into the zip.
3. Install via AEM Package Manager, then preview + publish.

Environments

Environment	URL Pattern	Purpose
Local	http://localhost:3000	Development (aem up --pages-url=...)
Feature Branch	https://{branch}-iggroup-agnesl-adobe.aem.page	PR testing
Preview	https://main-iggroup-agnesl-adobe.aem.page	Staging
Live	https://main-iggroup-agnesl-adobe.aem.live	Production

Git Workflow

Branch Naming

- Features: feature/description
- Fixes: fix/description
- Keep names short (branch appears in preview URLs — length limits apply)

PR Requirements

1. Include the preview URL in the PR description
2. Pass linting (`npm run lint`)
3. Test on the feature-branch preview
4. No console errors
5. If models changed, ensure `npm run build:json` output is committed

Troubleshooting

Block not loading?

1. Check the block folder name matches the class name in the DOM.
2. Verify the JS exports `default function decorate`.
3. Check the Network tab for 404s on the block JS/CSS.
4. For custom blocks, confirm the block is present in `component-filters.json` (rebuild via `build:json`).

Local preview shows wrong / no content?

- This is a repoless org — you must pass `--pages-url=https://main--iggroup--agnesl-adobe.aem.page` to `aem up`.

Brand font not applying?

1. Confirm `brand.css @font-face` URLs (`a.c-dn.net`) are reachable.
2. Matter SQ uses `font-display: swap` — a fallback (arial) shows briefly; that is expected.

columns-* variant styling missing in AEM?

- Expected known issue: forked columns blocks serialize to native `columns` without a variant class in JCR. Re-model as a `classes` option on base `columns`. See `jcr/README.md`.

Universal Editor dialog missing fields / block not allowed?

1. Edit the source in `models/_component-*.json` (not the root files).
2. Run `npm run build:json` and redeploy.

Content not updating?

1. Confirm the JCR page is installed/authored in AEM at `/content/iggroup/language-masters/en`.
2. Trigger preview/publish via `admin.hlx.page`.
3. Wait for CDN cache (1-2 min).

Resources

Resource	URL
Edge Delivery Services Documentation	https://www.aem.live/docs/
Universal Editor / Crosswalk	https://www.aem.live/developer/component-model-definitions
Developer Tutorial	https://www.aem.live/developer/tutorial
Block Collection	https://www.aem.live/developer/block-collection
E-L-D Loading	https://www.aem.live/developer/keeping-it-100
Best Practices	https://www.aem.live/docs/dev-collab-and-good-practices
helix-md2jcr	https://github.com/adobe/helix-md2jcr

Support Contacts

- Config Service org admin: agnesl@adobe.com
- Repository: <https://github.com/agnesl-adobe/iggroup>